P5

Mindy Kim, Woody Hulse

Time Spent:

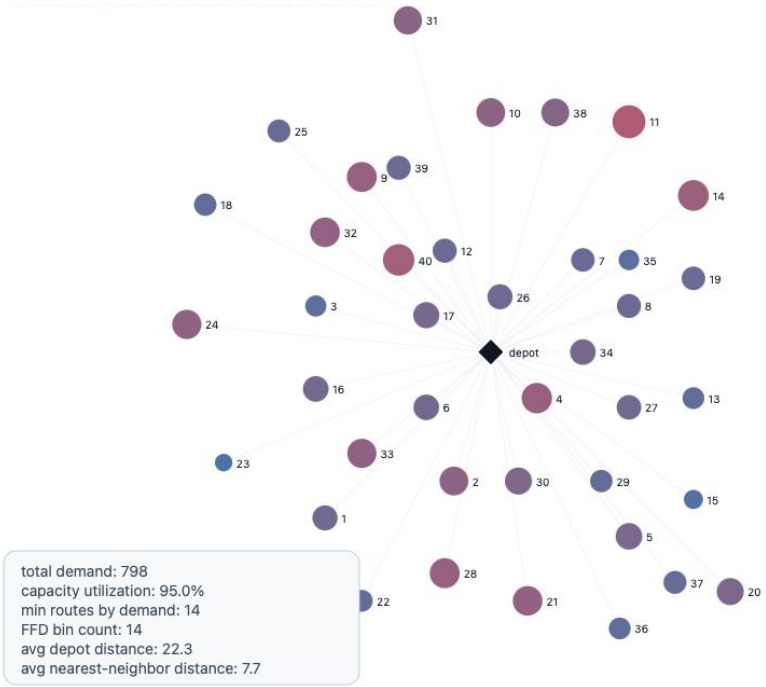
Optimizing

our humble attempts

		Baseline	+ Local search		+ LNS	
Instance	n	Obj.	Obj.	Δ (%)	Obj.	Δ (%)
101_11_2	100	3,513.40	3,377.71	-3.9	2,033.18	-42.1
101_8_1	100	854.46	839.42	-1.8	835.20	-2.3
121_7_1	120	1,121.19	1,098.12	-2.1	1,096.21	-2.2
135_7_1	134	1,318.22	1,256.54	-4.7	1,168.49	-11.4
151_15_1	150	4,935.84	4,733.23	-4.1	3,261.39	-33.9
16_5_1	15	451.24	352.16	-22.0	352.16	-22.0
200_16_2	199	2,588.53	2,367.86	-8.5	1,701.51	-34.3
21_4_1	20	364.45	364.45	+0.0	362.41	-0.6
241_22_1	240	766.04	745.24	-2.7	743.91	-2.9
262_25_1	261	10,111.48	9,757.72	-3.5	7,527.30	-25.6
30_4_1	29	506.55	505.01	-0.3	505.01	-0.3
386_47_1	385	46,819.65	44,603.97	-4.7	33,611.23	-28.2
41_14_1	40	1,395.41	1,045.95	-25.0	898.33	-35.6
45_4_1	44	757.67	730.53	-3.6	727.75	-3.9
51_5_1	50	534.13	524.61	-1.8	524.61	-1.8
76_8_2	75	786.39	763.36	-2.9	758.55	-3.5
Total		76,824.67	73,065.89	-4.9	56,107.24	-26.9

our final (?) solver

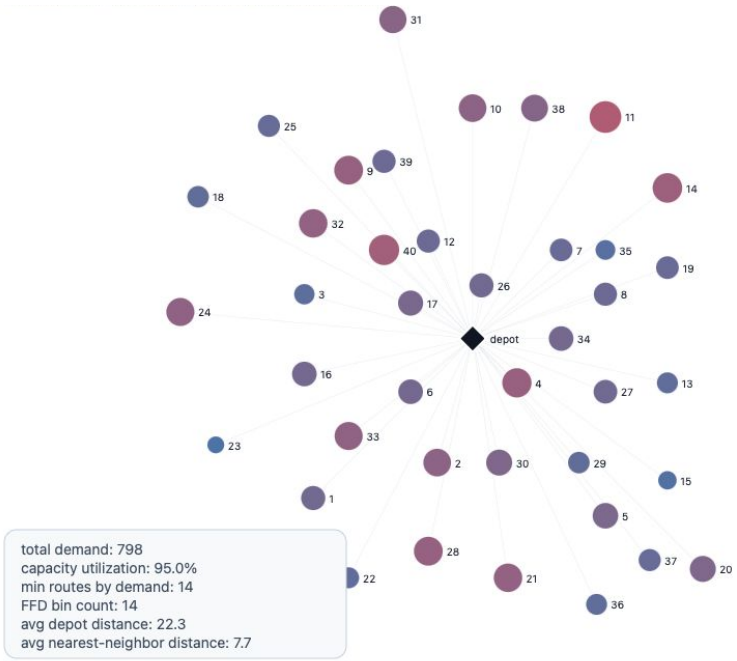
1. **Seed the population**
    a. Build starting solutions (nearest-neighbor + split, sweep clustering, hybrid, greedy packing)
2. **Main Search Loop**
    a. Hybrid Genetic Search x Large Neighborhood Search
    b. If stuck, bigger LNS or partially restart
3. **Final Search**
    a. Do small local search on best solution found from main loop



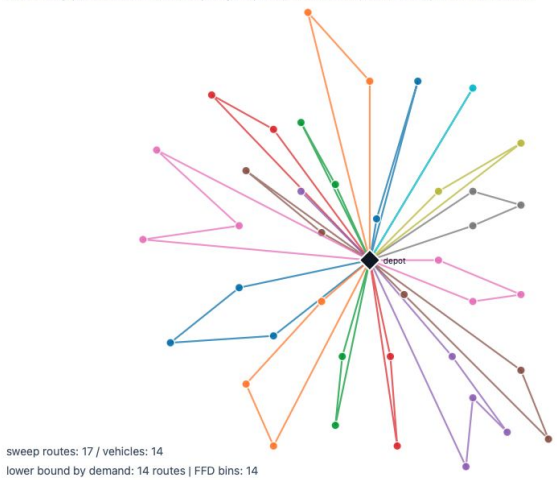
Hybrid Genetic Search (HGS)	Large Neighborhood Search (LNS)

● Maintains population of candidate solutions and evolves over time
● Pick two "parent" solutions, crossover to make a child, repeat
  ○ Children get educated before being judged
● Diversity > quality (doesn't collapse into one region of possibilities)
	● Destroy-and-repair
	  ○ Take current best solution
	  ○ Rip out substantial chunk of it (say, 25–55% of customers)
	  ○ Put customers back in (i.e. greedily, adaptive)
	● Find different arrangements that small moves couldn't have reached

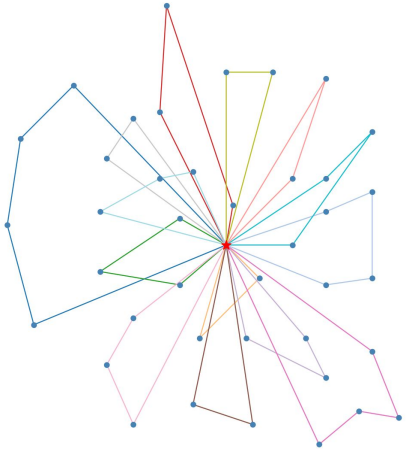
results!



Instance 41_14_1



Sweep Algorithm



Our Solver

“Optimizing”

graph flow matching

Training

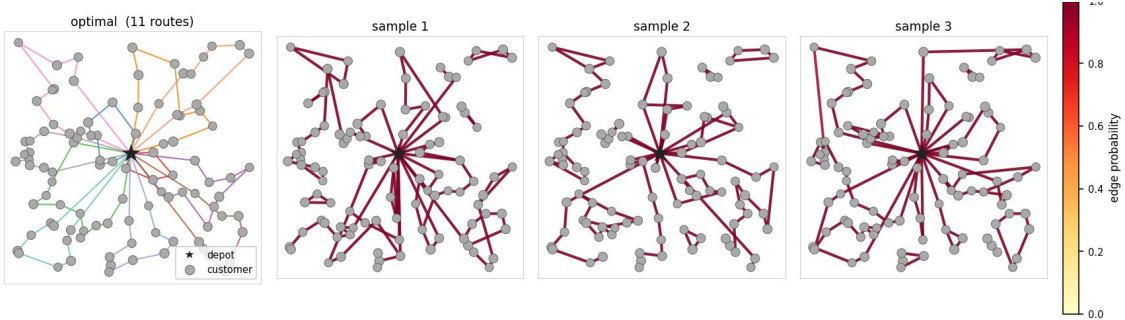
- Generate 1000 synthetic .vrp files
- Label graphs with optimized model
- Train FM model on these problems
  - Model iteratively denoises the edges (routes)

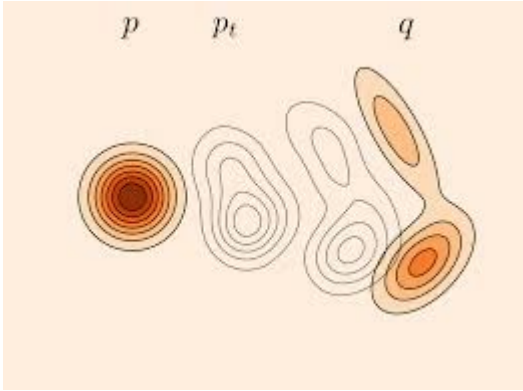
Routing

- Sample k times from N(0, 1)^d and pass through FM
- FM samples are seeds for giant tour split

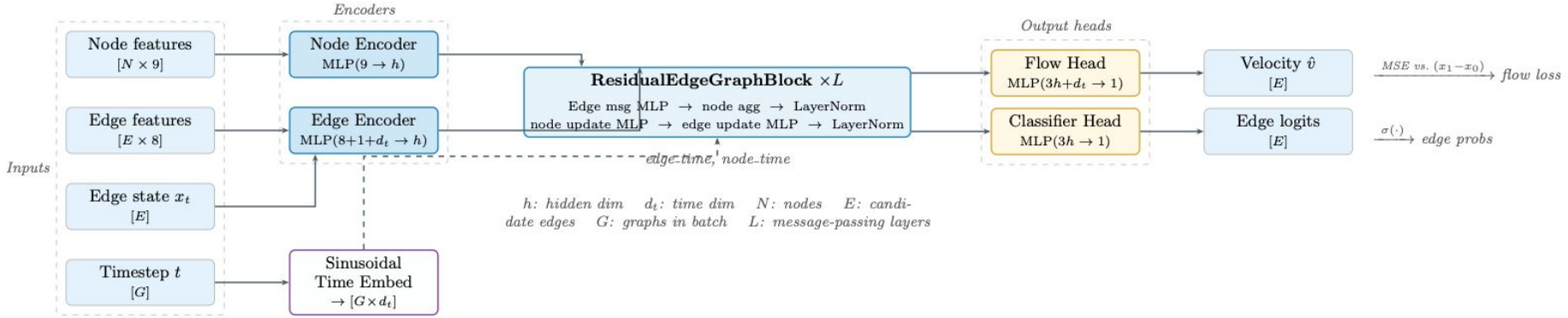
+ LNS		+ FM	
Obj.	Δ (%)	Obj.	Δ (%)
2,033.18	-42.1	2,033.18	-42.1
835.20	-2.3	828.84	-3.0
1,096.21	-2.2	1,131.21	+0.9
1,168.49	-11.4	1,167.30	-11.4
3,261.39	-33.9	3,261.39	-33.9
352.16	-22.0	345.36	-23.5
1,701.51	-34.3	1,701.51	-34.3
362.41	-0.6	358.40	-1.7
743.91	-2.9	732.43	-4.4
7,527.30	-25.6	7,527.30	-25.6
505.01	-0.3	505.01	-0.3
33,611.23	-28.2	33,611.23	-28.2
898.33	-35.6	898.33	-35.6
727.75	-3.9	723.54	-4.5
524.61	-1.8	529.17	-0.9
758.55	-3.5	744.24	-5.4
56,107.24	-26.9	55,098.44	-28.3

Flow matching





graph flow matching



fungus and slime (gross)

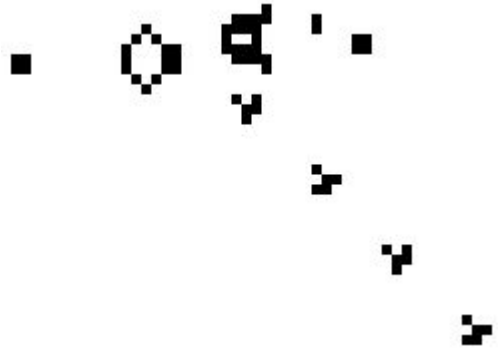
“Cellular automata”

- Use Euclidean-ness of the plane to define a grid and local update rules
- Can find “low-energy” paths

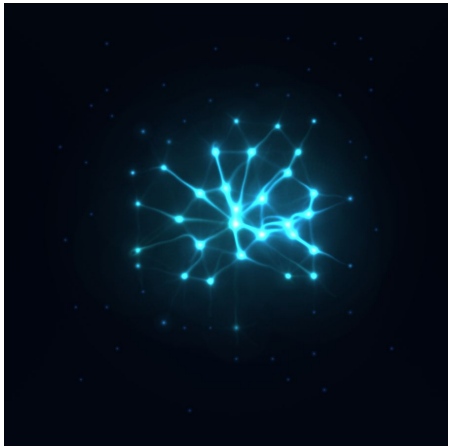
Slime

- Swarms of individual agents make local decisions

Idea: Seed the solver with slime routes

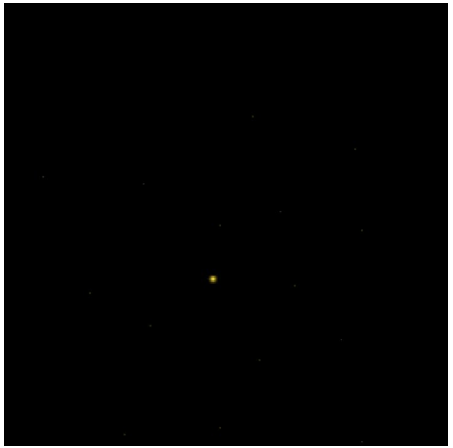


“Conway’s Game of Life” CA



Slime simulation in 76_8_2

Fungus solving a maze



Fungus simulation in 16_5_1

questions?